



Department of Computer Science

UET Lahore, New Campus

Registration Nos:

Complex
Problem

Computing

CS-272 Design and Analysis of
Algorithms

Deadline:

12th June 2026

Total Marks: 15

Marks Obtained:

Note:

- You can work in a group of 4, not less.
- One assignment per group is required.
- The deadline for this problem is 12th June 2026.
- Every member of the group must be present at the time of evaluation.
- Vivas will be conducted after 12th June 2026.

=====

Complex Computing Problem

The Integrated Logistics & Value-Optimization Problem

No.	Attributes	Description
WP1	Depth of Knowledge Required	This CCP demands a deep understanding of algorithm complexity theory, combinatorics , calculating solution space sizes, designing recurrence relations, and advanced optimization design patterns .
WP2	Range of Conflicting Requirements	This core conflict lies in maximizing the financial/priority "Priority Value" while simultaneously minimizing the physical distance/travel cost of a Hamiltonian path. Maximizing value pushes the system to select higher-value packages. However, doing so rapidly exhausts the finite weight capacities and time budgets of the couriers. Adding an item might yield a higher priority value but force a massive detour in routing, making the entire sub-route invalid. The dynamic extension introduces a conflict between maintaining a globally "optimal" static plan and reacting instantly to a high-priority real-time interruption via an event-driven heuristic.
WP7	Interdependence	The three core components i.e., Assignment (courier to depot), Selection (0/1 knapsack mapping), and Routing (TSP) cannot be solved in isolation. If a courier is assigned to a specific depot (Assignment), it alters the entire distance matrix. The subset of items chosen (Selection) strictly dictates which nodes <i>must</i> be visited during the pathing stage (Routing). If the routing stage determines that an item choice violates the time budget, the system must backtrack and change the selection or assignment altogether. The phases depend entirely on one another to reach validity.

[Mapping CLOs: CLO5: 5 CLO4: 3 CLO3: 2]

Assessment Rubrics:

Assessment Component	Exemplary (100%)	Proficient (75%)	Developing (50%)	Unacceptable (0%)
Complexity Analysis	Correctly establishes mathematical parameters that collapse the problem into TSP, 0/1 Knapsack, and GAP. Formulates an accurate solution space size and a comprehensive recurrence relation, detailing exact intractability.	Identifies all three reductions but lacks precision in setting rigid parameter constraints. Recurrence relation or solution space math has minor structural errors.	Misses one or more problem reductions. The recurrence relation is vague or does not accurately represent an exhaustive search.	Fails to outline reductions; mathematical formula for the solution space is absent or entirely incorrect.
Exact Solver via Branch and Bound	Provides clear, robust logic for fractional greedy upper bounding and MST lower bounding. Successfully demonstrates a state-space execution trace showing a clear branch prune using and optimal value bounds.	Logic for upper (fractional) and lower (MST) bounding is present but lacks conceptual clarity. The backtracking trace is generic and lacks clear data variables.	Only one bounding method (Upper or Lower) is accurately defined. The backtracking conditional check is flawed or missing execution steps.	No state-space pruning mechanism explained or implemented; no trace provided.
Heuristic Selection and Analysis	Implements a flawless localized DP sub-solver with the explicit recurrence equation for tracking i and w . Correctly proves the local time complexity is $O(N \cdot Q_k)$.	DP sub-solver is functional, but the explicit recurrence equation contains minor variable misalignments. Complexity proof is brief but mostly correct.	The DP recurrence equation is incomplete. The sub-problem does not cleanly separate from the global problem, violating the $O(N \cdot Q_k)$ boundary.	No DP sub-solver or recurrence relation is provided. Complexity analysis is missing.
Dynamic Extension	Designs a logically sound, event-driven decision rule for mid-route package insertion ($v_{\text{new}}, w_{\text{new}}$). Adequately accounts for spatial re-routing costs and courier budget violations.	Designs a decision rule for real-time changes but overlooks edge cases (e.g., ignoring remaining capacity or path limits while evaluating the new item).	The decision rule is overly simple (e.g., accept purely if v_{new} is high) without factoring in real-time geographic or constraint shifts.	No dynamic adjustment strategy or event-driven rule is presented.
System Flow Diagram and Deliverables	System execution flow diagram is highly detailed, mapping out assignment, selection, routing, and the dynamic pipeline clearly. All parts answered thoroughly.	Flow diagram covers all core stages but lacks granular detail on dynamic data loops or backtracking conditions. Answers are complete but brief.	Flow diagram is fragmented or conceptually misrepresents the interdependence of components. Deliverables are partially missing.	Flow diagram is missing or unreadable; critical deliverables are completely left out.

Problem Statement:

You are designing an autonomous delivery system for a high-value electronics hub. You have K couriers and N potential delivery tasks. Each task is located at a specific coordinate and has a "Priority Value" (v_i) and a "Package Weight" (w_i). Unlike a standard routing problem, you cannot fulfill all orders; you must select the most valuable ones while respecting courier constraints.

Formal Components

- Assignment Component:** You must assign each of the K couriers to a specific starting "Zone" or "Depot" to minimize the initial mobilization cost.
- Selection Component:** Each courier has a weight capacity (Q_k). You must select a subset of items for each courier that maximizes the total Priority Value ($\sum v_i$) without exceeding (Q_k).
- Routing:** Once items are selected and assigned, each courier must find the shortest Hamiltonian path (starting and ending at their assigned depot) to deliver those items.

The Core: This integrated problem tests your ability to handle deeply conflicting requirements, maximizing abstract financial or priority profit while simultaneously minimizing physical travel expenditures.

Evaluative Tasks and Questions

Part 1: Complexity Analysis (2 Marks)

Objective: Prove your deep understanding of asymptotic boundaries and complex problem classification.

- **Question 1.1:** State exactly how this integrated problem collapses into:
 1. The Traveling Salesperson Problem
 2. The 0/1 Knapsack Problem.
 3. The Generalized Assignment Problem. Define the strict parameter constraints (e.g., values of distance, weight, or K) required for each reduction.
- **Question 1.2:** Calculate the exact theoretical size of the solution space combining item selection (2^N), courier assignment (K^N), and path permutations ($N!$). Write the comprehensive recurrence relation $T(N)$ representing an **Exhaustive Search** strategy and explain why this becomes mathematically intractable as N grows.

Part 2: Exact Solver via Branch and Bound (3 Marks)

Objective: Design and implement a deterministic framework that guarantees an optimal solution for medium-scale inputs.

- **Question 2.1:** Develop and document the logic for your state-space tree pruner:
 1. Explain how you calculate the **Upper Bound** for item selection using a fractional value-per-weight greedy approach.
 2. Explain how you calculate the **Lower Bound** for routing costs using a Minimum Spanning Tree approximation.
- **Question 2.2:** Show how your backtracking algorithm abandons a branch. What conditional check forces a backtrack when the remaining potential value drops below your current optimal value, or when estimated MST distance violates a courier's max time budget (T_k)? Provide a trace step from your execution by demonstrating a successful prune.

Part 3: Heuristic Selection & Analysis (3 Marks)

Objective: Analyze the computational trade-offs between exact optimization and rapid heuristic execution under conflicting constraints.

- **Question 3.1:** Implement a localized Dynamic Programming sub-solver to address individual courier capacity constraints (Q_k).
 1. Write the explicit DP recurrence equation used to track the maximum value across an item subset i and weight limit w .
 2. Show that this local sub-problem runs within $O(N \cdot Q_k)$ time complexity.

Part 4: Dynamic Extension (2 Marks)

Objective: Extend the static system architecture to accommodate real-time, unpredictable environmental shifts.

- **Question 4.1:** Modify your system to handle a **Dynamic Logistics** scenario. Suppose that while a courier is mid-route executing their computed TSP path, a new high-priority order (v_{new}, w_{new}) suddenly appears at a specific coordinate.
 1. Design an event-driven decision rule: How does the system determine whether to interrupt the courier's active path to pick up the new package, or reject the order completely?

Deliverables:

- ✓ A complete system execution flow diagram.
- ✓ The formal answers to every question.